

# Dataform Repo Tour — a Real Silver→Gold Repository

A guided reading of the public demo repository from the Day 1 walkthrough (3:10–3:25 PM CT): [github.com/jwdavis/dataform-demo](https://github.com/jwdavis/dataform-demo). Browse it in your browser — no setup needed. This is what a real Silver→Gold repo looks like; your CI/CD instincts (git, PRs, release configs) all apply.

## What the repo builds

A small three-layer medallion in BigQuery — the same shape as your target architecture:

| Layer          | Files                                                                                                | Lands in dataset       | Your architecture                                                 |
|----------------|------------------------------------------------------------------------------------------------------|------------------------|-------------------------------------------------------------------|
| Raw (declared) | definitions/<br>customers_declaration.sqlx,<br>orders_declaration.sqlx,<br>products_declaration.sqlx | dataform_demo_raw_data | <b>Bronze</b> — landed by Datastream/loads (this morning's story) |
| Staging        | definitions/<br>staging/<br>stg_customers.sqlx,<br>stg_orders.sqlx,<br>stg_products.sqlx             | dataform_demo_staging  | <b>Silver</b> — cleaned, typed, filtered                          |
| Marts          | definitions/marts/<br>daily_sales.sqlx,<br>customer_metrics.sqlx,<br>product_performance.sqlx        | dataform_demo_marts    | <b>Gold</b> — the marts Power BI points at                        |

Nine nodes total in the compiled graph. Bronze to Gold is six small SQL files.

## Read it in this order

### 1. dataform.json — the project settings

Verbatim from the repo:

```
{  
  "warehouse": "bigquery",  
  "defaultSchema": "dataform_demo",  
  "assertionSchema": "dataform_assertions",  
  "defaultDatabase": "<project-id>",  
  "defaultLocation": "US"  
}
```

One JSON file says it all: the warehouse is BigQuery, which GCP project to build in (defaultDatabase — <project-id> is a placeholder the repo owner replaced), the default dataset, and where assertion results live. Compare that with forty packages each burying their own connection strings.

## 2. includes/constants.js — names in one place

A short JavaScript file exporting constants: the project ID and the three dataset names (DATASET\_RAW, DATASET\_STAGING, DATASET\_MARTS). Every sqlx file references these constants instead of repeating literals — the same reason you keep environment configuration out of package XML.

## 3. definitions/\*\_declaration.sqlx — Bronze enters the graph

The raw tables are created and loaded outside Dataform — by Datastream, by scripts, by anything. A declaration tells Dataform "this table exists and is managed elsewhere; let models ref() it":

```
// definitions/customers_declaration.sqlx
config {
  type: "declaration",           // tells Dataform this is an externally managed table
  schema: constants.DATASET_RAW, // the dataset where customers actually lives
  name: "customers"
}
```

This is how your Bronze layer — landed by the migration tooling — becomes a first-class node in the dependency graph instead of an invisible upstream assumption.

## 4. definitions/staging/stg\_\*.sqlx — Silver

Each staging file is type: "table" plus cleaning SQL you could write in your sleep — TRIM, LOWER, PARSE\_DATE, null filters. The first 14 lines of stg\_customers.sqlx (a WHERE clause filtering null email/name follows in the file):

```
config {
  type: "table",
  schema: constants.DATASET_STAGING,
  description: "Cleaned customer data"
}
```

### SELECT

```
customer_id,
TRIM(customer_name) as customer_name,
LOWER(TRIM(email)) as email,
PARSE_DATE('%Y-%m-%d', registration_date) as registration_date,
UPPER(TRIM(country)) as country,
CURRENT_TIMESTAMP() as processed_at
FROM ${ref("customers")}
```

The one thing SSIS never gave you: `{ref("customers")}` points at the declared raw table, and from that single function Dataform derives the execution order. No precedence constraints to drag around a designer surface.

## 5. definitions/marts/\*.sqlx — Gold

The marts are aggregate joins over the staging models: `daily_sales` (orders, revenue, completed/cancelled counts per day), `customer_metrics` (lifetime value, order counts, lifespan per customer), `product_performance`. Each is type: "table" in the marts dataset, selecting from `{ref("stg_orders")}` and friends. This is the layer Power BI will query.

## What the repo deliberately does not have: assertions

The repo ships no assertion files — data-quality tests are something you add. The shape of one, dropped into a staging model's config block (this is the snippet added live in the walkthrough):

```
assertions: {  
  uniqueKey: ["customer_id"],  
  rowConditions: ['email IS NOT NULL']  
}
```

Assertions run with the pipeline and fail the run — the "how do we know the data's good" question, answered where the data is made. You will add one yourself in the lab if you finish early.

## Also in the repo

`misc/` holds the CSV load scripts and a Terraform configuration that provisions the GCP side (Dataform repository, git connection, service account, datasets). Browse it if you are curious — you do not need to run any of it for class. After class, the repo README walks through forking it and provisioning your own copy with Terraform: a good first exercise for whoever prototypes back home.

## Why this matters for your migration

- The compiled graph you saw — declarations → staging → marts, edges drawn from `ref()` calls — **is** the Silver→Gold pipeline as code. Reviewable in a pull request, versioned in git.
- Development workspace = feature branch. Release configuration = your deployment. Scheduled invocation = SQL Agent. The same discipline you run for SQL Server deployments today, applied to SQL in git.
- Constants in one file, datasets declared once, execution order derived — nothing to babysit between "merge" and "tables rebuilt."

## Self-check

- ☐ I can explain what `dataform.json` and `includes/constants.js` each own.
- ☐ I can explain why the raw tables are *declared* rather than *created* in this repo.
- ☐ I can trace one path from a declaration to a mart (`customers` → `stg_customers` → `customer_metrics`) and name the Bronze/Silver/Gold layer of each node.

- I could write an assertion block for "order\_id unique, quantity > 0" without looking it up.

## Next

Build the same bones yourself in the [Dataform lab](#) (3:25–4:05 PM CT): repo, workspace, sqlx, execute.

- Docs: [Dataform overview](#) · [Assertions](#)
- Repo: <https://github.com/jwdavis/dataform-demo>

*Part of the Iowa DOM DoIT Data Engineering student pack — see ../00\_schedule.md.*